

Detailed AI Learning Reference: RAG + LangGraph for Python/Kubernetes Engineers

This is the deeper reference sheet for your 7-day plan. It explains the concepts, the flow, and the exact tasks you should complete.

0. Beginner setup: get your environment ready

This section is for a total beginner. Follow these steps in order.

Step 1: create your project folder

```
mkdir -p ~/ai-learning-project  
cd ~/ai-learning-project
```

Step 2: create a virtual environment

```
python3 -m venv .venv  
source .venv/bin/activate  
python -V
```

If you are on Windows PowerShell:

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1
```

Step 3: upgrade pip

```
python -m pip install --upgrade pip
```

Step 4: install the main AI packages

Use one of these depending on the model provider you want:

For OpenAI:

```
pip install openai langgraph langchain chromadb python-dotenv
```

For Anthropic:

```
pip install anthropic langgraph langchain chromadb python-dotenv
```

If you want a local embedding model too:

```
pip install sentence-transformers
```

Step 5: create a `.env` file

```
cat > .env <<'EOF'
OPENAI_API_KEY=your_key_here
# or ANTHROPIC_API_KEY=your_key_here
EOF
```

Then load it in Python:

```
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

Step 6: create your first Python file

```
cat > basic_llm.py <<'PY'
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain Kubernetes readiness probes in 3 sentences."},
    ],
    temperature=0.2,
)

print(response.choices[0].message.content)
PY
```

Run it:

```
python basic_llm.py
```

Step 7: verify you understand the result

A successful response means:

- your environment works
- your API key works
- you can call an LLM from Python

If this fails, the problem is usually:

- missing environment variable
- wrong API key
- package not installed
- model name invalid

Useful beginner docs

- OpenAI Python quickstart: <https://platform.openai.com/docs/quickstart>
- Anthropic Python docs: <https://docs.anthropic.com/>
- LangGraph docs: <https://langchain-ai.github.io/langgraph/>
- Chroma docs: <https://docs.trychroma.com/>

1. Foundations: What is an LLM call?

At the core, an LLM call is just:

- input text or structured messages
- model decides next tokens
- returns generated output

Typical message structure:

```
messages = [  
    {"role": "system", "content": "You are a helpful assistant."},  
    {"role": "user", "content": "Explain Kubernetes readiness probes."}  
]
```

Critical idea:

- The model does not understand memory like a human brain
- It predicts the next token based on patterns and context
- The quality of output depends heavily on prompt design, examples, and context

Tasks

- Write a Python script that calls the model
- Test three prompts: simple, more specific, constrained
- Compare outputs

What to remember

- prompt quality matters
- system messages guide behavior

- context window matters

2. What is RAG?

RAG stands for Retrieval-Augmented Generation.

The idea is:

1. Find relevant documents or chunks
2. Retrieve the most relevant pieces
3. Add them to the prompt
4. Ask the LLM to answer based on that context

Basic RAG flow

```
User question
-> search in vector DB
-> top-k relevant chunks
-> add to prompt
-> LLM generates answer grounded in docs
```

Why RAG matters

Without RAG, the model may answer from general knowledge. With RAG, it can answer based on your docs, internal data, or domain knowledge.

Core pieces

- document loading
- chunking
- embeddings
- vector store
- retrieval
- prompt assembly

Beginner step-by-step: build a minimal RAG app

Step 1: create a docs folder

```
mkdir -p data/docs
```

Add a few text files, for example:

- `data/docs/kubernetes\_deployments.md`
- `data/docs/readiness\_probe.md`

- `data/docs/pod\_lifecycle.md`

Each file should contain a few paragraphs of plain text about the topic.

Step 2: install more packages

```
pip install chromadb
```

Step 3: write a simple RAG script

```
from pathlib import Path
import os
from dotenv import load_dotenv
from openai import OpenAI
import chromadb

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# 1. load documents
texts = []
for path in Path("data/docs").glob("*.md"):
    texts.append({"id": path.stem, "text": path.read_text()})

# 2. simple split: one chunk per file
chunks = [item["text"] for item in texts]

# 3. create a Chroma db
client_db = chromadb.Client()
collection = client_db.create_collection(name="k8s_docs")

# 4. embed each chunk using a simple placeholder approach
# In real code, use an embedding model. For now, we just keep it simple.
# You will usually do:
# embeddings = client.embeddings.create(model="text-embedding-3-small", input=chunks)

# 5. the user question
question = "What does a readiness probe do in Kubernetes?"

# 6. in a real app, you would retrieve top-k chunks here
# then pass them into the prompt
retrieved_text = "A readiness probe checks if a container is ready to receive traffic. If it fails, Kubernetes"

prompt = f"""
Use the following context to answer the question.
If the answer is not in the context, say you do not know.

Context:
{retrieved_text}

Question: {question}
"""

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful Kubernetes assistant."},
        {"role": "user", "content": prompt},
    ],
    temperature=0.2,
)

print(response.choices[0].message.content)
```

This is a simplified example. Real RAG uses embeddings to find relevant text automatically.

Step 4: test the script

```
python rag_basic.py
```

What to learn from this

- retrieval is just searching for relevant context
- the LLM is told to answer using that context
- the model is more grounded when it sees the real docs

3. Embeddings and vector stores

Embeddings are numeric representations of text.

They allow you to compare semantic similarity:

- question embedding
- document embedding
- nearest neighbors retrieval

Example flow

```
question = "What is a readiness probe in Kubernetes?"
embedding = model.encode(question)
results = vector_store.search(embedding, k=5)
```

Vector stores like Chroma or FAISS help find the closest text chunks.

Beginner command to install a vector DB

```
pip install chromadb
```

Real embedding example with OpenAI

```
from openai import OpenAI

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.embeddings.create(
    model="text-embedding-3-small",
    input=["What is a readiness probe?", "How does Kubernetes route traffic?"],
)

print(response.data[0].embedding[:5])
```

Tasks

- Create 5–10 text docs
- Split them into chunks
- Embed chunks with a model
- Query the vector DB with a question
- Check whether the retrieved docs are related

Common issue

Bad chunking leads to poor retrieval. Too large = noisy. Too small = loses context.

4. Chunking strategy

This is a practical topic you should learn early.

Good chunk size

A common starting point:

- 250–800 tokens per chunk
- overlap of 20–80 tokens

Why overlap matters

It preserves continuity between chunks and helps with related concepts across boundaries.

Example

If a Kubernetes doc covers:

- Deployment
- ReplicaSet
- Pod lifecycle

A chunk that spans those concepts may retrieve better than a tiny isolated sentence.

Beginner task

Open a doc and split it manually into 3–5 chunks. Then ask yourself:

- Is each chunk self-contained?
- Does it preserve context?
- Is information lost at boundaries?

Minimal script for chunking

```
text = "A readiness probe checks if a container is ready to receive traffic. If it fails, Kubernetes does not  
chunks = text.split(". ")  
print(chunks)
```

This is not production-grade, but it teaches the idea.

5. Prompt engineering for RAG

After retrieval, you normally build a prompt like this:

```
prompt = f"""  
Use the following context to answer the question.  
If the answer is not in the context, say you do not know.  
  
Context:  
{retrieved_text}  
  
Question: {question}  
"""
```

This matters because:

- it keeps the model grounded
- it reduces hallucination
- it avoids unsupported claims

Good prompt patterns

- define the role
- specify the task
- instruct the model to use only retrieved context
- say when to refuse or admit uncertainty

Beginner prompt test

```
prompt = """  
You are a Kubernetes support assistant.  
Use only the provided context. If the answer is not in the context, answer: 'I don't know based on the provided context.'  
  
Context:  
{context}  
  
Question:  
{question}  
"""
```

Tasks

- Write 3 versions of the prompt
- Compare answer quality

- Check which one is most grounded and concise

6. LangGraph fundamentals

LangGraph is a framework for building stateful, graph-based LLM applications.

Core concepts

- State: data shared across nodes
- Node: a function or step in the pipeline
- Edge: transition from one node to another
- Conditional edge: choose the next step based on state or model output

Example graph

```
User question
-> router
    -> retrieve if needed
    -> answer
```

This is useful because you can model real decision logic instead of a single prompt pipeline.

What the graph does

It gives you a repeatable workflow:

1. input arrives
2. decide on next step
3. perform action
4. update state
5. continue until complete

Install LangGraph

```
pip install langgraph
```

Minimal LangGraph example

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    question: str
    answer: str

builder = StateGraph(State)

def router(state: State):
```

```

    question = state["question"]
    if "kubernetes" in question.lower() or "pod" in question.lower():
        return {"answer": f"Need retrieval for: {question}"}
    return {"answer": f"Direct answer for: {question}"}

builder.add_node("router", router)
builder.add_edge(START, "router")
builder.add_edge("router", END)

app = builder.compile()

result = app.invoke({"question": "What is a readiness probe?"})
print(result)

```

Tasks

- Build a graph with 3 nodes: `router`, `retrieve`, `answer`
- Pass a state dictionary between nodes
- Ensure the result includes question + retrieved context + final answer

7. Agent patterns in LangGraph

An agent is a loop where the model can decide to:

- call a tool
- inspect results
- decide what to do next
- stop when complete

Typical loop

```

User asks question
-> model decides to use tool
-> tool runs
-> result added to state
-> model reasons again
-> answer or another tool call

```

Good beginner tools

- search across docs
- calculator
- date/time lookup
- shell command (careful in production)

Beginner tool example

```

def add(a: float, b: float) -> float:
    return a + b

```

```
print(add(2, 3))
```

This is not yet an LLM tool, but it teaches the idea of a callable function that the agent can use.

Task: create a small tool

```
def kubernetes_lookup(term: str) -> str:
    lookup = {
        "readiness probe": "A readiness probe checks if a container is ready to receive traffic.",
        "liveness probe": "A liveness probe checks if a container is still alive.",
    }
    return lookup.get(term.lower(), "No direct match found.")

print(kubernetes_lookup("readiness probe"))
```

Tasks

- Create 2 small tools
- Add them to a LangGraph agent
- Trigger a tool call and inspect the output
- Confirm the model can continue reasoning after the tool result

8. Domain-specific project suggestions

Since you know Python and Kubernetes, choose a project with domain value.

Best beginner use cases

1. Kubernetes docs assistant Questions like:

- What is the difference between a Deployment and a StatefulSet?
- What does a readiness probe do?
- How do I debug a CrashLoopBackOff pod?

2. Internal platform docs assistant Use internal Markdown docs, runbooks, and architecture documents.

3. Kubernetes + GitOps support bot Answer questions about YAML, Helm, ArgoCD, Kustomize, or GitOps workflows.

Why these are strong

- aligns with your background
- easy to test with realistic questions
- good demo material
- demonstrates real world value

9. Project architecture to build

Your first project can be structured like this:

```
project/
├── app/
│   ├── main.py
│   ├── rag.py
│   ├── graph.py
│   └── tools.py
├── data/
│   └── docs/
│       └── sample_questions.md
├── .env
├── requirements.txt
├── README.md
└── notebooks/
    └── quick_experiments.ipynb
```

Responsibilities

- `main.py`: entry point
- `rag.py`: vectors, retrieval, chunking
- `graph.py`: LangGraph flow
- `tools.py`: available functions for the agent

Initial file creation commands

```
mkdir -p app data/docs notebooks
cat > requirements.txt <<'EOF'
openai
langgraph
langchain
chromadb
python-dotenv
EOF
```

Then create files:

```
touch app/main.py app/rag.py app/graph.py app/tools.py README.md
```

10. Practical tasks to complete every day

Daily checkpoint tasks

- Write one script or small module
- Run it locally

- Inspect the output
- Save notes on what worked and what failed
- Keep the code small and understandable

Keep it focused

A good daily result is not “learn everything”; it is:

- working code
- one concept understood
- one result validated

11. Common pitfalls to avoid

Pitfall 1: Start with complex frameworks

Do not start with a huge app or heavy orchestration. Start with raw API + simple RAG.

Pitfall 2: Bad docs for retrieval

If your documents are messy or too long, retrieval will feel broken even if the model is fine.

Pitfall 3: Overloading the prompt

Do not include too much context. Keep it relevant.

Pitfall 4: Treating the LLM as magic

It is a system that follows patterns and context. Understand the flow.

Pitfall 5: Trying to optimize too early

First: make it work. Then tune retrieval. Then add tools. Then improve prompts.

12. Learning resources

Must-read / must-use

- OpenAI API docs
- Anthropic docs
- LangGraph docs
- Chroma docs
- FAISS docs

- LangChain docs (use as reference, not your first base)

Good learning approach

- read 20–30 minutes of docs
- build 30–60 minutes of code
- test + small notebook experiments
- summarize what you learned in notes

Example command to start with docs locally

```
python -m http.server 8000
```

Then open the docs in a browser if needed. For AI learning, the official docs are generally more reliable than random tutorials.

13. Example mini-project idea for your week

Project: Kubernetes companion assistant

Functions:

- search Kubernetes docs / notes
- answer conceptual questions
- explain deployment lifecycle
- offer next troubleshooting step

Sample questions:

- What is the difference between a Deployment and a DaemonSet?
- Why is my pod stuck in CrashLoopBackOff?
- What does a liveness probe do?
- How do I debug a failed readiness check?

This is ideal because it blends:

- RAG
- LangGraph
- domain expertise
- practical value

Quick starter command for sample project setup

```
mkdir -p data/docs
cat > data/docs/kubernetes_basics.md <<'EOF'
A Deployment manages a set of replicas for stateless apps.
A StatefulSet manages stateful workloads with stable identities.
A readiness probe checks whether a container is ready to receive traffic.
A liveness probe checks whether a container is alive.
EOF
```

14. Final learning objective

By the end of the week, you should be able to explain these clearly:

- What an LLM call is
- What embeddings are
- What vector retrieval is
- What RAG is
- What a graph-based workflow is
- What an agent does
- Why tools and state matter

That is a strong base to build from.

15. Quick “done” checklist

Before you consider this week successful, verify:

- ☐ You can call an LLM from Python
- ☐ You built a basic RAG flow
- ☐ You built a LangGraph flow
- ☐ You built an agent with tool calling
- ☐ You have a working mini-project
- ☐ You can explain how it works in plain English

If all of the above are true, you are in a strong starting position.

16. Fast-start checklist for your first 3 days

Day 1 goal

- Create venv
- Install packages
- Run a basic LLM call
- Save the script

Day 2 goal

- Create a docs folder
- Load docs
- Build a simple retrieval prompt
- Run a sample question

Day 3 goal

- Install LangGraph
- Build a state graph
- Create router and answer nodes
- Run a simple workflow

This is the minimum working knowledge needed to move to full project work.

3. perform action
4. update state
5. continue until complete

Tasks

- Build a graph with 3 nodes: `router`, `retrieve`, `answer`
- Pass a state dictionary between nodes
- Ensure the result includes question + retrieved context + final answer

7. Agent patterns in LangGraph

An agent is a loop where the model can decide to:

- call a tool
- inspect results
- decide what to do next
- stop when complete

Typical loop

User asks question
-> model decides to use tool
-> tool runs
-> result added to state
-> model reasons again
-> answer or another tool call

Good beginner tools

- search across docs
- calculator
- date/time lookup
- shell command (careful in production)

Tasks

- Create 2 small tools
- Add them to a LangGraph agent
- Trigger a tool call and inspect the output
- Confirm the model can continue reasoning after the tool result

8. Domain-specific project suggestions

Since you know Python and Kubernetes, choose a project with domain value.

Best beginner use cases

1. Kubernetes docs assistant Questions like:

- What is the difference between a Deployment and a StatefulSet?
- What does a readiness probe do?
- How do I debug a CrashLoopBackOff pod?

2. Internal platform docs assistant Use internal Markdown docs, runbooks, and architecture documents.

3. Kubernetes + GitOps support bot Answer questions about YAML, Helm, ArgoCD, Kustomize, or GitOps workflows.

Why these are strong

- aligns with your background
- easy to test with realistic questions
- good demo material
- demonstrates real world value

9. Project architecture to build

Your first project can be structured like this:

```
project/
├── app/
│   ├── main.py
│   ├── rag.py
│   ├── graph.py
│   └── tools.py
├── data/
│   └── docs/
│       └── sample_questions.md
├── .env
├── requirements.txt
├── README.md
└── notebooks/
    └── quick_experiments.ipynb
```

Responsibilities

- `main.py`: entry point
- `rag.py`: vectors, retrieval, chunking
- `graph.py`: LangGraph flow
- `tools.py`: available functions for the agent

10. Practical tasks to complete every day

Daily checkpoint tasks

- Write one script or small module
- Run it locally
- Inspect the output
- Save notes on what worked and what failed
- Keep the code small and understandable

Keep it focused

A good daily result is not “learn everything”; it is:

- working code
- one concept understood
- one result validated

11. Common pitfalls to avoid

Pitfall 1: Start with complex frameworks

Do not start with a huge app or heavy orchestration. Start with raw API + simple RAG.

Pitfall 2: Bad docs for retrieval

If your documents are messy or too long, retrieval will feel broken even if the model is fine.

Pitfall 3: Overloading the prompt

Do not include too much context. Keep it relevant.

Pitfall 4: Treating the LLM as magic

It is a system that follows patterns and context. Understand the flow.

Pitfall 5: Trying to optimize too early

First: make it work. Then tune retrieval. Then add tools. Then improve prompts.

12. Learning resources

Must-read / must-use

- OpenAI API docs
- Anthropic docs
- LangGraph docs
- Chroma docs
- FAISS docs
- LangChain docs (use as reference, not your first base)

Good learning approach

- read 20–30 minutes of docs
- build 30–60 minutes of code
- test + small notebook experiments
- summarize what you learned in notes

13. Example mini-project idea for your week

Project: Kubernetes companion assistant

Functions:

- search Kubernetes docs / notes
- answer conceptual questions
- explain deployment lifecycle
- offer next troubleshooting step

Sample questions:

- What is the difference between a Deployment and a DaemonSet?
- Why is my pod stuck in CrashLoopBackOff?
- What does a liveness probe do?
- How do I debug a failed readiness check?

This is ideal because it blends:

- RAG
- LangGraph
- domain expertise
- practical value

14. Final learning objective

By the end of the week, you should be able to explain these clearly:

- What an LLM call is
- What embeddings are
- What vector retrieval is
- What RAG is
- What a graph-based workflow is
- What an agent does
- Why tools and state matter

That is a strong base to build from.

15. Quick “done” checklist

Before you consider this week successful, verify:

- ☐ You can call an LLM from Python
- ☐ You built a basic RAG flow
- ☐ You built a LangGraph flow
- ☐ You built an agent with tool calling
- ☐ You have a working mini-project
- ☐ You can explain how it works in plain English

If all of the above are true, you are in a strong starting position.